

Lab-Test 2: MathPuzzle.

Program:

```
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class MathPuzzleGame {
    private static final String Questions-Files = "questions-Files";
    private static final String score-File = "score-history.txt";
    static class Question {
        int operand1;
        char operator;
        int operand2;
        int correctAnswer;
```

```

public Question(int op1, char operator, int op2, int ans) {
    this.operand1 = op1;
    this.operator = operator;
    this.operand2 = op2;
    this.correctAnswer = ans;
}

public String getQuestionString() {
    return operand1 + " " + operator + " " + operand2 + "=?";
}

public static void main(String[] args) {
    Scanner consoleScanner = new Scanner(System.in);

    String playerName;
    String difficultyStr;
    int difficultyMultiplier = 1;
    int totalScore = 0;

    System.out.println("=====");
    System.out.println("Welcome to the Math Puzzle!");
    System.out.println("=====");

    System.out.print("Enter your name: ");
    playerName = consoleScanner.nextLine();

```

```

boolean validDifficulty = false;
while (!validDifficulty) {
    System.out.print("choose difficulty (easy, medium, hard): ");
    difficultyStr = consoleScanner.nextLine().toLowerCase();
    switch (difficultyStr) {
        case "easy":
            difficultyMultiplier = 1;
            validDifficulty = true;
            break;
        case "medium":
            difficultyMultiplier = 2;
            validDifficulty = true;
            break;
        case "hard":
            difficultyMultiplier = 3;
            validDifficulty = true;
            break;
        default:
            System.out.println("Invalid selection. Please type  

            'easy', 'medium', or 'hard'.");
    }
}

```



```
System.out.println("\nStarting quiz for " + playerName + " Good  
Luck!\n");
```

```
try {
```

```
    List<Question> questionList = loadQuestionsFromFile(questi-  
ons-File);
```

```
    if (questionList.isEmpty()) {
```

```
        System.out.println("Error: no Questions found in " + Questions-File);  
        return;
```

```
    }
```

```
    int questionCount = 1;
```

```
    for (Question q: questionList) {
```

```
        System.out.print("Q" + questionCount + ": " + q.getQuestionString());
```

```
        while (!consoleScanner.hasNextInt()) {
```

```
            System.out.println("That's not a number! Try again.");
```

```
            System.out.print("Q" + questionCount + ": " + q.getQuestionString());
```

```
            consoleScanner.next();
```

```
            int playerAnswer = consoleScanner.nextInt();
```

```
            if (playerAnswer == q.correctAnswer) {
```

```
                System.out.println("Correct! (" + " + difficultyMultiplier + " points);
```

```
                totalScore += difficultyMultiplier;
```

```
            } else {
```

```
                System.out.println("Wrong. The answer was " + q.correctAnswer);
```

```
            }
```

```

        questionCount++;
    }
    System.out.println("n = = = = = = = = = = = = = = = = ");
    System.out.println("Game over");
    System.out.println("n = = = = = = = = = = = = = = = = ");

    saveScoreFile(playerName, totalScore, difficultyMultiplier);
} catch (FileNotFoundException e) {
    System.err.println("Critical Error: The input file " +
        QuestionsFile + " was not found.");
    System.err.println("Please ensure questions.txt is in  
the project root directory.");
} catch (IOException e) {
    System.err.println("An error occurred while performing  
file I/O: " + e.getMessage());
} finally {
    consoleScanner.close();
}
}

```

```

private static List<Question> loadQuestionsFromFile
(String fileName) throws FileNotFoundException {

```

```

    List<Question> questions = new ArrayList<>();

```



```

File file = new File(fileName);
Scanner fileScanner = new Scanner(file);
while (fileScanner.hasNextLine()) {
    String line = fileScanner.nextLine();
    Scanner lineScanner = new Scanner(line);
    if (lineScanner.hasNextInt()) {
        int op1 = lineScanner.nextInt();
        char operator = lineScanner.next().charAt(0);
        int op2 = lineScanner.nextInt();
        int ans = lineScanner.nextInt();
        questions.add(new Question(op1, operator, op2, ans));
    }
    lineScanner.close();
}
fileScanner.close();
return questions;
},

private static void saveScoreToFile(String name,
int score, int multiplier) throws IOException {
    try (BufferedWriter writer = new BufferedWriter(
        new FileWriter(scoreFile, true))) {
        DateTimeFormatter dff = DateTimeFormatter.ofPattern(
            "yyyy/MM/dd HH:mm:ss");

```

```
LocalDateTime now = LocalDateTime.now();  
String difficultyLabel = (multiplier == 1)?  
    "Easy":(multiplier == 2)? "Medium": "Hard";  
String scoreEntry = String.format("[%s] Player:  
%s - %s | Difficulty: %s - %s | Score: %s",  
    diff.format(now), name, difficultyLabel, score);  
writer.write(scoreEntry);  
writer.newLine();  
System.out.println("Score successfully saved to "  
    SCORE_FILE);  
}  
}
```